楼层业态类型字段实施计划

背景

PropOS 管理的楼栋包含综合体（`property_type = 'mixed'`），其中不同楼层可能分属写字楼、商铺、公寓三种业态。当前 `floors` 表和 `Floor` 模型均无 `property_type` 字段，导致混合体楼栋的楼层无法在楼层级别标注业态，进而影响：

- 单元创建时的业态继承
- 楼层热区的业态色彩渲染
- DXF 导入后的楼层业态配置
- Excel 批量导入单元时的业态来源

核心决策

决策点	选择
楼层业态是否强制必填	**可选**，非 mixed 楼栋自动从楼栋继承；mixed 楼栋可为 NULL 后续补填
单元业态更新策略	**后端静默级联**：楼层业态变更时，同一事务内更新该楼层所有未归档单元的 `property_type`
DXF 导入时业态设置入口	在 CadImportDialog **指派未匹配 SVG 的表格行**中增加楼层业态下拉
Excel 导入业态来源	新增**楼层业态列**（Col 2），单元 `property_type` 从楼层继承，不再有独立业态列

Phase 1 — 数据库（文件：`backend/migrations/`）

001_add_property_type_to_floors.sql（增量迁移，用于已有数据库）

```
ALTER TABLE floors ADD COLUMN IF NOT EXISTS property_type property_type;

-- 回填：非 mixed 楼栋的楼层继承楼栋业态
UPDATE floors f
SET property_type = b.property_type
FROM buildings b
WHERE f.building_id = b.id
  AND b.property_type IN ('office', 'retail', 'apartment');

CREATE INDEX IF NOT EXISTS idx_floors_property_type
  ON floors(property_type) WHERE property_type IS NOT NULL;
```

000_consolidated_schema.sql（全新建库，同步加入该列）

在 `CREATE TABLE floors` 定义内加入：

```
-- 楼层所属业态（混合体楼栋需逐层指定；非混合体楼栋自动继承楼栋业态）
```

```
property_type   property_type,
```

Phase 2 — 后端 Floor 模型 + Repository

改动文件

- `backend/lib/modules/assets/models/floor.dart`
  - `Floor` 类新增 `final String? propertyType;`
  - `fromColumnMap` 读 `map['property_type'] as String?`
  - `toJson` 加 `'property_type': propertyType`
- `backend/lib/modules/assets/repositories/floor_repository.dart`
  - `findAll` / `findById` SELECT 语句增加 `f.property_type::TEXT AS property_type`
  - `create()` 新增 `String? propertyType` 参数，INSERT 时写入该列
  - 新增 `updatePropertyTypeWithCascade(String floorId, String propertyType)` 方法：
    - 同一事务：`UPDATE floors SET property_type=@pt WHERE id=@id`
    - 同一事务：`UPDATE units SET property_type=@pt WHERE floor_id=@id AND archived_at IS NULL`
    - 返回 `int`（级联更新的单元数量）

Phase 3 — 后端 Floor Service

改动文件：`backend/lib/modules/assets/services/floor_service.dart`

- `createFloor()` 增加 `String? propertyType` 参数：
  - 若楼栋 `property_type != 'mixed'`：忽略传入值，自动使用楼栋业态
  - 若楼栋 `property_type == 'mixed'`：使用传入值（可 null）
- 新增 `patchFloor(String id, {String? propertyType, String? floorName, double? nla})` 方法：
  - 若 `propertyType != null`：调 `repo.updatePropertyTypeWithCascade`，返回 `updatedUnitCount`
  - 其余字段走普通 `UPDATE floors SET ...`
  - 返回 `({Floor floor, int updatedUnitCount})`

Phase 4 — 后端 Floor Controller

改动文件：`backend/lib/modules/assets/controllers/floor_controller.dart`

- `POST /api/floors`（`_create`）：body 增加可选 `property_type` 字段，传给 service
- 新增路由 `PATCH /api/floors/<id>`（`_patch`）：
  - 读取 body 中 `property_type?`、`floor_name?`、`nla?`
  - 调 `service.patchFloor`
  - 响应格式：`{ "data": { ...floor, "updated_unit_count": N } }`

响应结构示例

```
PATCH /api/floors/uuid
{
  "property_type": "office"
}
→ 200 { "data": { "id": "...", "property_type": "office", ..., "updated_unit_count": 12 } }
```

Phase 5 — 后端 CAD Import

改动文件

- `backend/lib/modules/assets/services/cad_import_service.dart`
  - `assignUnmatched()` 新增可选 `String? propertyType` 参数
  - 指派 SVG 到楼层后，若 `propertyType != null` ，调 `floorService.patchFloor(floorId, propertyType: propertyType)`
- `backend/lib/modules/assets/controllers/cad_import_controller.dart`
  - `PATCH .../assign` handler 从 body 读取可选 `property_type` 字段，透传给 service

Phase 6 — 后端 Excel 单元导入

改动文件：`backend/lib/modules/assets/services/unit_import_service.dart`

新模板列映射（15 列）：

列号	字段	说明
0	楼栋名称	解析为 building_id
1	楼层名称	解析为 floor_id
2	楼层业态（新增）	写入 `floors.property_type` ，单元继承
3	单元编号	unit_number
4-14	其余单元字段	与旧模板 Col 3-13 对应

逻辑规则：

- 同楼层内所有行的楼层业态必须一致，不一致时报 `FLOOR_PROPERTY_TYPE_CONFLICT`
- 同楼层首次遇到时：若楼层已有 `property_type` 且与 Excel 值不一致，报同一错误码
- 单元 `property_type` 从楼层继承，不再从 Excel 行读取
- 旧 13 列模板兼容： `header[2]` 不是「楼层业态」时，自动回退旧解析逻辑（Col 2 = 单元编号，Col 3 = 业态）

Phase 7 — Admin 前端

改动文件

`admin/src/types/asset.ts`

- `Floor` 接口增加 `property_type?: PropertyType | null`

`admin/src/api/modules/assets.ts`

- 新增 `patchFloor(id, payload)` 函数（`PATCH /api/floors/:id`），返回 `Floor & { updated_unit_count: number }`
- `assignUnmatched(jobId, svgLabel, floorId, propertyType?)` 增加可选 `property_type` 参数
- `createFloor` payload 增加可选 `property_type?: PropertyType`

`admin/src/views/assets/components/CadImportDialog.vue`

- Step 4 未匹配 SVG 表格增加「楼层业态」列（`<el-select>`，选项：写字楼/商铺/公寓）
- 指派时同步传 `property_type` 给 API

`admin/src/views/assets/BuildingDetailView.vue`

- 楼层列表 `<el-table>` 增加「业态」列，渲染 `<el-tag>`（语义色）
- 行内点击业态 Tag → 弹出 `<el-select>` 下拉修改 → 自动调 `patchFloor`
- 成功后 ElMessage 提示「已更新楼层业态，同步更新了 N 个单元」
- mixed 楼栋时才显示业态列和编辑功能（非 mixed 楼栋的楼层业态固定显示楼栋业态，不可编辑）

Phase 8 — Flutter 端

改动文件

`flutter_app/lib/features/assets/domain/entities/floor.dart`

- `Floor` entity 增加 `PropertyType? propertyType` 字段（`@freezed`）

`flutter_app/lib/features/assets/data/models/floor_model.dart`

- `FloorModel` 增加 `@JsonKey(name: 'property_type') String? propertyType` 字段
- `toEntity()` 传入 `propertyType: propertyType == null ? null : PropertyType.fromString(propertyType)`

Phase 9 — uni-app 端

改动文件：`app/src/types/assets.ts`

- `Floor` 接口中 `property_type` 字段类型由 `PropertyType`（当前或缺失）调整为 `PropertyType | null`
- 无 UI 改动（`pages/assets/floors.vue` 已按业态渲染徽章）

验证步骤

1. 数据库：`psql` 执行 `001_add_property_type_to_floors.sql`，确认 `\d floors` 含 `property_type` 列，存量记录已回填

2. **后端编译**：`cd backend && dart analyze` 无错误
3. **后端测试**：`dart test test/` 中 floor_service 级联逻辑用例通过
4. **Admin 类型检查**：`cd admin && pnpm type-check` 无错误
5. **Admin UI**：打开综合体楼栋详情，确认楼层列表有「业态」列，可点击修改，提示更新单元数
6. **Admin CadImportDialog**：DXF 切分完成 → 指派未匹配 SVG 时，业态下拉可选且传参正确
7. **Admin 单元导入**：用新 15 列模板测试，楼层 `property_type` 正确写入，单元继承楼层业态
8. **Flutter**：楼层列表/详情页 `propertyType` 字段不报解析错误

边界说明

- **不强制** mixed 楼栋的楼层必须填业态（保持可选，NULL 代表待定）
- **不支持** 单元个别覆盖楼层业态（单元业态始终与楼层保持一致）
- **不涉及** Phase 2 以外功能（租户门户、门禁、电子签章等）
- uni-app `floors.vue` 已有业态徽章渲染，本次无 UI 改动